

冲刺回顾是团队分析根因在下一轮冲刺改进的最佳时机，前面介绍了时间表 (Timeline)，颜色点等方法收集团队“软”数据，这章会针对“硬”数据。如果团队了解改正 (Correction) 和纠正措施 (Corrective Action) 的区别，和根因分析，便可用在回顾时分析数据，制定有效纠正措施，避免发生过的问题再发生。

收集哪些数据

美国某家医院的故事

1996 年，某家美国 Cook County 医院做了个试验。这家医院所在的地区比较穷困，很多市民没有买医疗保险，且医院床位紧张。因人口老化，也没有良好饮食习惯，这医院面对的最大问题是怎么判断病人有没有急性心脏病的危险？

因为如果病人胸口痛，医生从他的背景、年龄、生活习惯判断有心脏病的危险，就要收他住院，但医院的床位有限。新来的院长 Reilly 很有经验，了解问题严重，首先要解决这个难题，因为针对心脏病的治疗的病床要花费 2000 美元一晚，一般心脏病要住 3 晚，而后面发现很多病人其实没有心脏病，浪费了医院的宝贵资源。他发现每个医生都会有自己的一套方式去判断病人是否需要住院，诊断的差异很大。每个医生要收集的数据也很多，心电图、血液、尿液、x 光，还有病人的历史、习惯等。因为院长是从东岸名牌医院过来的。他认识专家专门研究怎么有效去判断病人的专家。这位专家 Goldman 用了两年时间，针对心脏病猝死的诊断，收集了很多数据和验证，发现只需要针对 3 个紧急的风险因素，就可以准确判断：

1. 病人的肺部有没有液体
2. 血压上压 (systolic Blood pressure) 是否低于 100
3. 病人是否感到心绞痛 (patient unstable angina)

通过这样就可以判断病人是否需要住院。所以院长就依据这个做了实验。结果发现平常用以前的方式，医生诊断准确度只有 75%-85%，但用了这个准确度就超过 95%。

很多人都误解度量分析必须像大数据分析一样，要有大量数据，用很多因子来求出一个漂亮的方程式，才算成功。其实是反过来，如果想要收集的数据量越大变量越多，量化过程改进越难成功，开发人员越抵触。所以我们要针对想改进的重点收集对应数据。

很多时候信息量多，但是没有针对性，反而不好。

好比病人去医院，都必须抽血、验尿、做心电图，收集各类数据让医生诊断，但是信息越多越齐全就越好。从上面美国 Cook County 医院的故事看到其实不是。小时候看病，中医是用望闻问切的方式；西医就是用听诊器听心跳、呼吸，看舌头、口腔，然后也是听你的问题描述，便可以诊断开药了。现在去医院排队看病，医生第一件事就给你开四五张各类的检查单，通过这些检查报告做诊断。虽然我是外行，但我觉得小时候那种不超过十分钟诊断过程不会比现代的起码两个小时的诊断质量差。

如何建立并完善缺陷预测模型的例子

公司度量分析人员展示了一个大表，上面密密麻麻有五、六十个项目变量，包括缺陷、进度、工作量等等。

我问：为什么你们要收集这么多数据量？你们的改进目标是什么？

度量分析人员：我们要提高生产率、减少进度偏差，也要降低最终的客户的缺陷密度。

我问：如能尽早发现缺陷，加强前面的评审效率来减少遗漏到后面的缺陷，减少返工。如果大家都相信这可行，就应该专注这模型相关的度量，如每迭代的缺陷分布、与相关返工工作量等，让蒙地卡罗预测模型可以尽快跑起来。但如果对进度偏差，工作量偏差等都要收集，每轮可能要收集几十个变量，反而没有针对性。我们看看这预测缺陷模型，比如你们有什么不同的评审方式？效果有什么不同？

度量分析人员：我们跟开发讨论过，确实有些区分，例如，如果我们在前期有做正式评审，质量都会好一些，但是如果没的话，后面确实缺陷就多了。如果我们有明确需求，例如按活动、实体方式明确写出来，跟没有的话也会有质量的区分。有些项目有规定要静态扫描代码并记录缺陷，后期的缺陷数会较少。

我总结说：所以如果可以估计各种方式的缺陷排除率/引入缺陷率，便可使用模型预测最终会遗漏多少缺陷。可利用这蒙地卡罗预测工具，帮你比较各种方法，制定缺陷数目标。团队开始时先参考上一轮迭代数据，估一下预测模型参数，并一起头脑风暴讨论有什么新方法可以做提升，加进模型，蒙地卡罗预测模型可比较不同的组合，看看对缺陷率，缺陷排除率和工作量等有什么影响，团队便可参考模型的建议，最终选择下一迭代的配搭。例如有些团队利用静态代码扫描预先找出代码缺陷，就可以做个实验，区分一下是否代码扫描后，系统测试缺陷会减少，排除缺陷的百分比是多少等。开始的时候可能数据量不多，慢慢项目多了数据也可以看是否稳定，便开始变成团队的基线。所以开始时不要太计较准不准，而是尽快利用这概念开始量化管理。

如果团队能尽早发现缺陷并修复，后面返工量便可倍数降低，生产率自然也会提升，进度偏差也会改善。所以建议只针对能如何尽早发现并解决缺陷，减少返工工作量设定目标，量化管理。

缺陷排除率

从需求到设计、编码、单元测试、系统测试、验收，整个开发过程大家都很熟悉，需求、设计、编码后可以通过评审来排除缺陷，但仅仅做评审不一定能确保质量。因为最终验收缺陷数取决于每个步骤能否有效排除当前的缺陷。

可用缺陷排除率 (Defect Removal Efficiency DRE) 来判断每一个过程的质量：

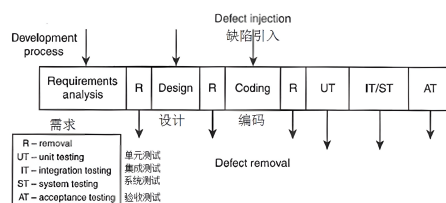


图 13-1

$$DRE = \frac{\text{Defects found by the QC activity 缺陷排除数}}{\text{Total defects in the product before QC 缺陷总数}}$$

如果要最终质量好，缺陷排除率就要高。但计算缺陷排除率，必须要等到整个开发完成才可以计算。

缺陷排除预测模型实例

- 团队分析迭代缺陷数据，缺陷是源自需求/设计/编码，得出以下分布表：

	Req	Design	Coding
RR	1		
DR	1	2	
CR	2	3	6
UT	2	5	10
ST	3	9	16
AT	1	1	2
	10	20	34

图 13-2

A/按缺陷排除率 (DRE) 公式，计算出各个过程的缺陷排除率，并把参数输进水晶球模型。例如第一行 Quality 那行：在底栋填 10 (共有 10 个缺陷是源自需求)；第二行：在第 4 栋填 10% (需求评审缺陷排除率是 10%)。(本来迭代缺陷参数都标 Infosys 识别。)

RR	1/10	10%
DR	1+2/(10+20-1)	10.3%
CR	2+3+6/(64-4)	18.3%
UT	2+5+10/(64-15)	34.7%
ST	3+9+16/(64-32)	87.5%
AT	1+1+2/(64-60)	100%

图 13-3

B/ 按以上迭代回顾缺陷分析结果，按以下输入框位置输进模板

- 1. 需求引入缺陷数 =10 (3)
- 2. 需求评审缺陷排除率 = 10% (4)
- 3. 设计引入缺陷数 =20 (2)
- 4. 设计评审排除率 =10% (4)
- 5. 代码引入缺陷数 =34 (4)
- 6. 代码评审排除率 =18% (1)
- 7. 单元测试缺陷排除率 =35% (2)
- 8. 系统测试缺陷排除率 =88% (1)
- 9. 验收测试缺陷排除率 =99% (3)

注：以上括号里的数字表示模板哪一栋，例如第一行需求引入是（3）代表在第三栋填 10。

	(1)	(2)	(3)	(4)
需求	缺陷数	缺陷数	缺陷数	缺陷数
需求评审	缺陷数	缺陷数	缺陷数	缺陷数
设计	缺陷数	缺陷数	缺陷数	缺陷数
设计评审	缺陷数	缺陷数	缺陷数	缺陷数
代码	缺陷数	缺陷数	缺陷数	缺陷数
代码评审	缺陷数	缺陷数	缺陷数	缺陷数
单元测试	缺陷数	缺陷数	缺陷数	缺陷数

表 13-1

	① 修复缺陷 总工时	② 缺陷数	① / ②
RR+DR	16	4	4
CR+UT	80	28	2.9
ST	160	28	5.8
AT	80	4	20

图 13-4

C/从团队工时表估算各过程的缺陷返工工作量。例如最下面一行 Quality 那行输入：\$20,000 (因为验收测试缺陷平均修复工时是 20，假定每工时成本为 \$1,000)；系统测试那行：\$5,800(因为系统测试缺陷平均修复工时是 5.8)。

Zone 4 AE2-AL60						
Requirements Development		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$71,888	\$905	\$700	\$800	\$1,000
	Quality					
Reqs Review		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$8,416	\$905	\$700	\$800	\$1,000
	Quality	\$16,736		\$3,400	\$4,000	\$4,600
Design		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$67,683	\$905	\$700	\$800	\$1,000
	Quality					
Design Review		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$6,308	\$704	\$700	\$800	\$1,000
	Quality	\$27,602	\$1,418	\$3,400	\$4,000	\$4,600
Code		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$88,801	\$904	\$700	\$800	\$1,000
	Quality					
Code Review		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$11,682	\$905	\$700	\$800	\$1,000
	Quality	\$21,898	\$1,417	\$2,486	\$2,800	\$3,326
Unit Test		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$144,168	\$905	\$700	\$800	\$1,000
	Quality	\$83,814	\$1,418	\$2,486	\$3,400	\$4,600
Integration Test		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$4	\$905	\$700	\$800	\$1,000
	Quality	\$3	\$1,418	\$3,400	\$3,400	\$4,600
System Test		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$78,604	\$905	\$700	\$800	\$1,000
	Quality	\$124,792	\$1,404	\$4,600	\$4,600	\$8,470
Acceptance Test		Costs	Unit Costs	Min	Most Likely	Max
Effort		\$62,786	\$904	\$700	\$800	\$1,000
	Quality	\$131,687	\$1,418	\$17,000	\$20,000	\$22,000

图 13-5

D/ 质量总成本：水晶球会依据质每过程的缺陷分布和单位成本分布，预估质量成本的分布。

质量总成本 + 工作（如评审，设计、开发、测试等）成本（注），得出总成本分布。

注：指具体做评审、测试等活动的工作量，填在模板里工时 Effort 那行
从水晶球得出总成本分布的 95 百分位数 (95% percentile) 为：\$905,022

E/ 团队分析根因后，团队找出以下 3 条改进建议：

1. 用正式专家评审加强需求评审，估计能把缺陷排除率从本来 10% 提高到 80%
 2. 用评审检查单加强设计评审，估计能把缺陷排除率从本来 10% 提高到 59%
 3. 用检查单加强代码评审，估计能把缺陷排除率从本来 18% 提高到 35%
- 问题 1：用了这些新方法会对缺陷分布有什么改善效果？
 - 问题 2：是否 3 种改善都使用？或只用其中 2 种更有效？（总成本最低）

把 3 种新方法和% 按下表加入模板：水晶球会依据质每过程的缺陷分布和单位成本分布，预估质量成本的分布。

1. 按本来迭代需求引入缺陷数 =10 (3)
2. 需求评审缺陷排除率 = **80%** (3)，需求评审缺陷排除率 = 10% (4)
3. 设计引入缺陷数 =20 (2)
4. 设计评审排除率 =**59%** (3)，设计评审排除率 =10% (4)
5. 代码引入缺陷数 =34 (4)
6. 代码评审排除率 =18% (1)，代码评审排除率 =**35%** (2)
7. 单元测试缺陷排除率 =35% (2)

- 8. 系统测试缺陷排除率 =88%（1）
- 9. 验收测试缺陷排除率 =99%（3）

最佳搭配是只选加强设计与代码评审，需求评审不变。
改进后成本分布的 95 百分位数 (95% percentile) 为：\$874,854

- 如果我们把优化后的缺陷分布与本来迭代的缺陷分布比较，能看出改进后，本来在系统测试才发现的缺陷大多数可以预先在评审暴露，降低返工工作量，也可以预估评审缺陷范围，如果实际评审缺陷数未能达到范围，便可预警，尽快纠正

如果没有预测模型，我们只能主观估计会降低，有了预测模型，便可以估计缺陷分布的变化，也能帮我们挑选最佳搭配，并预估下一轮的缺陷分布范围。

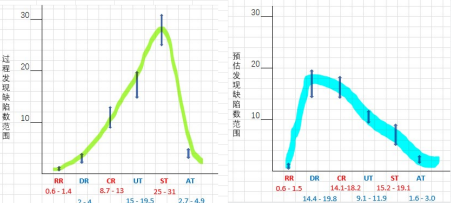


图 13-6

解读预测模型最佳搭配选择：
模型以总成本最低可以防止局部最优。例如需求评审没有选用排除率高的新方法，因为新方法会使用较大评审工作量。如果用质量最优（质量成本最低）便必然会读选用缺陷排除率最高的方法，但用总成本来比较便可以综合考虑工作量 (Effort) 和质量 (Quality)，避免局部最优。
由于不同项目甚至不同迭代冲刺的缺陷数和缺陷率各异，难以进行比较，但如果方法和团队人员相似，缺陷排除率应该是可比的。通过分析以往的缺陷数据，针对最薄弱的环节讨论根本原因，并在下一个迭代中进行改进，提升缺陷排除率，并可以预估各个阶段（如评审、测试等）的缺陷发现数。例如，如果需求评审的缺陷数远低于预期，就应立即进行纠正，而不是像以前那样仅仅提醒大家要重视评审工作。

== ==

怎样开始

一天的培训，可以帮助他们了解根因分析的重点。

如何提升软件开发质量TrainingPlan			
9:30	用案例介绍根因分析的五个步骤与技巧:帕累托图、鱼骨图等练习		
10:00	根因分析小组互动		
11:00	休息		
11:15	迭代回顾场景(由公司内部教练做角色扮演)		
11:45	总结,并介绍缺陷排除率模型		
12:00	午休		
13:00	介绍缺陷排除率模型		
13:15	量化根因分析模拟练习,包括: 分析模拟数据,估计缺陷排除率与返工工作量 针对主问题,团队所有成员做KJ根本原因分析 从根因、讨论对应改进措施		
15:15	休息		
15:30	设定量化质量计划模拟练习 利用MonteCarla工具 对原本迭代数据做3点估算 再依据前面制定的方案 利用工具寻找最佳方案		
16:15	总结,确认团队后面的具体工作		

图 13-7

让团队了解以下重点：

改进目标：降低返工工作量（同时也会降低缺陷后期才被发现的数量）

分析根因：不仅要分析后期发现缺陷的分类与数量，还需对从需求到最终客户验收的全生命周期进行缺陷排除率分析，以找出最薄弱的环节并深入挖掘根因。利用数据进行互动练习是最有效的学习方式，因此建议从以往的缺陷数据中抽取 70-150 条进行小组分析。

预测效果：以福特 8D 根因分析框架为例，缺陷排除与返工预测模型可以帮助团队预估新方法的效果（例如，通过代码静态扫描加强代码评审）。

团队在开始从定性到定量管理的时候，可以在回顾时，分析刚刚迭代的缺陷排除率和缺陷引入，以前没有这种细化的数据，根因分析可能只看到系统测试缺陷的总数多少，用了这个排除率的分解，就可以让团队了解主要问题来自哪里，针对性分析根因和对应措施。例如发现大部分的缺陷来自代码，就深入地去看代码出了什么问题，那一类代码缺陷比较多，针对这一类的缺陷讨论原因，下一轮做对应改进措施，并制定新一轮的量化目标。

量化回顾也让团队成员了解到自己的问题，更了解自己的质量和水平。

如何能简单有效地收集编码缺陷与返工工作量数据

最有效项目数据的方法就是每次迭代回顾时，同时看看本迭代缺陷的相关数据，这样团队立马可分析本迭代收集的数据（而非团队收集数据，让公司数据专员做分析）。很多团队说”我们都有评审代码，但问题都直接改正了，没有记录。“后面便不知道有多少缺陷，无法量化分析。如果要求开发人员按系统测试缺陷记录方式，在缺陷管理系统里面一条一条记，大家会觉得太花时间不愿意做。可以利用工具帮助，例如可利用 GIT 做代码评审记录，也可以在 GIT 系统里面跟踪评审问题。不需要把问题另外记在系统测试缺陷跟踪系统，但每条问题在 GIT 系统里都有痕迹。开发就可以从中数出大概共有多少代码评审缺陷。如果平常也记录了每天所花的时间，他用来修改 bug 的工作量也有统计数据了。团队自己收集数据：此外，团队成员应开始每天或每周自行统计返工的工作量（包括评审缺陷，所有数据均记录在缺陷管理系统中）。

缺陷 #	系统测试缺陷描述	开始	结束	总修复人时	备注
5384	当客户注册(即大陆身份证), 大陆地址指定, 系统报错, 无法继续	1/2/2010 - 1/2/2010	1/2/2010 - 1/2/2010	10	没有准备好各种设备, 资料尚未完善
5190	输入错误信息提醒, 但系统没有报错 (估计程序没有校验输入是否正确)	1/7/2010 - 1/8/2010	1/7/2010 - 1/8/2010	25	
5365	输入"96 99" 查询, 结果是阳性, 不对	1/1/2010 - 1/1/2010	1/1/2010 - 1/1/2010	5	内部错误, 但系统没有处理

表 13-2

建议每人自己每天记录当天工时花在什么地方，比如花在缺陷的修复的是多少。迭代结束回顾时，每人看看自己的记录。团队迭代回顾时，因为在系统里面也有评审缺陷的跟踪也可以数出来缺陷是多少，从而可以算到平均一个缺陷花多少返工工时。这样就不会要开发人员花太多时间做记录，但回顾还可以收集到开发过程的所需数据。

从迭代复盘到持续改进

当某些团队已经能做好回顾和根因分析，也能利用数据改善下一轮的开发质量时，如何将这种做法转变为团队或公司的持续改进呢？

深圳有一家公司专门从事内部 IT 服务，因为对质量要求严格，新版本在正式投产（发版）前必须通过测试验收。公司每两周进行一次发版，并对发版成功率进行统计。因为都做了很久，依据以往水平，要求发版成功率不会低于 93%，并一直都监控这个百分比趋势。某月，发现成功率跌到了 92%，这引发了质量改进组启动根因分析。他们首先确认是哪个部门的问题导致发版成功率下降。后来发现有两个部门的表现最差，于是针对这两个部门进行根因分析，也对一些原因实施了纠正措施，如培训和评审等。做以后发现确实那些问题解决了。通过这些措施，问题得以解决，发版成功率也提升了，到了 96%。后面他们把这个作为新的基线下限，提升至一个新的水平。

虽然个别项目组的迭代回顾能够有效改善质量，但这些成果仅仅是高层关注下的“种子”。尽管通过工作量和缺陷的改善能够显著减少持续浪费，但如果高层不关注并投入资源，那么这些“改善”可能只是短暂的闪光，很快大家就会回到以前的工作状态。反之，如公司把过程改进当成项目来管理，定目标，定期监控，便能像以上案例形成标杆（基线），而且会像上台阶一样逐步提升。注意：开始时，各团队水平不同，一般难以形成公司级标杆，但可以先形成项目组自己的标杆。

总结

有数据才可以谈改进，但收集数据花精力，所以必须有针对性，二八原则可以帮助识别应针对哪方面入手。软件开发应，软件开发团队花最多工作量的不是编码本身，用于修复缺陷工作量最多。软件开发有个特点，缺陷越后暴露，返工工作量越多，例如如果缺陷在客户使用后才暴露，可能总共要花 20~50 人时修正，但如果缺陷能在前面评审或单元测试发现和改正，就不会超过 1 人时。但很多团队，大部分缺陷都是在系统测试或验收时才暴露，前面评审、单元测试发现的缺陷都很少。虽然很多团队的根因分析用了帕里拖图、鱼骨图的方式，这些都是基于主观判

断的方式，没有真正利用基于事实的数据，而且很容易受从众心理影响，变成项目经理的一言堂，无法挖掘出真正原因，使用便利贴 KJ 方式可以帮团队，更好从事实总结出背后的主因。

团队所有成员一起，包括需求、开发、测试、项目经理等一起分析迭代的缺陷数据，可以计算迭代的缺陷注入和缺陷排除率，利用蒙地卡洛预测模型就可以把本来只是定性的一个改进目标变成定量，也可以满足 CMMI4 级的主要实践（详见附件）。

参考 Reference

1. Juran, Joseph. "Juran's Quality Handbook "(1999) 5/e
2. Gladwell, Malcolm. "Blink: The Power of Thinking Without Thinking "